

TASK 25: ASSIGNMENT NO. 25

Name of Student : Anuja Vilas Wankhade

Domain: Artificial intelligence and machine Learning (AIML)

College Name: Manav School of Polytechnic AKOLA

Branch : Computer Engineering

GitHub Repository: <https://github.com/anujaw193-star/python-project-collection>

1. (Manual Hyperparameter Tuning - KNN)

Code:

```
[1]  # Q1. Manual Hyperparameter Tuning - KNN
✓ 2s

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Load the Iris dataset
iris = load_iris()

# Independent Features (X) and Dependent Feature (y)
X = iris.data
y = iris.target

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.33,
    random_state=42
)

# Different values of n_neighbors
neighbors = [3, 5, 7, 11, 13, 15]

best_k = 0
best_accuracy = 0

# Train and evaluate for each value
for k in neighbors:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
```

```

# Train and evaluate for each value
for k in neighbors:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)

    accuracy = model.score(X_test, y_test)

    print(f"n_neighbors: {k} --> Test Accuracy = {accuracy:.4f}")

    if accuracy > best_accuracy:
        best_accuracy = accuracy
        best_k = k

print("\n-----")
print(f"Best n_neighbors : {best_k}")
print(f"Highest Accuracy : {best_accuracy:.4f}")

```

Output:

```

... n_neighbors: 3 --> Test Accuracy = 0.9800
    n_neighbors: 5 --> Test Accuracy = 0.9800
    n_neighbors: 7 --> Test Accuracy = 0.9800
    n_neighbors: 11 --> Test Accuracy = 1.0000
    n_neighbors: 13 --> Test Accuracy = 1.0000
    n_neighbors: 15 --> Test Accuracy = 1.0000

-----
Best n_neighbors : 11
Highest Accuracy : 1.0000

```

2. (Manual Hyperparameter Tuning - SVM)

Code:

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.33,
    random_state=42
)

# Hyperparameters to try
C_values = [1, 10, 20]
kernels = ['linear', 'rbf']

# Try all combinations of C and kernel
for c in C_values:
    for kernel in kernels:
        model = SVC(C=c, kernel=kernel)
        model.fit(X_train, y_train)

        accuracy = model.score(X_test, y_test)
        print(c, kernel, accuracy)

        print(f"C: {c}, Kernel: {kernel} --> Test Accuracy = {accuracy:.4f}")

        if accuracy > best_accuracy:
            best_accuracy = accuracy
            best_C = c
            best_kernel = kernel

print("\n-----")
print("Best Hyperparameters:")
print(f"Best C : {best_C}")
print(f"Best Kernel : {best_kernel}")
print(f"Best Accuracy : {best_accuracy:.4f}")

```

Output:

```
... 1 linear 1.0
C: 1, Kernel: linear --> Test Accuracy = 1.0000
1 rbf 1.0
C: 1, Kernel: rbf --> Test Accuracy = 1.0000
10 linear 1.0
C: 10, Kernel: linear --> Test Accuracy = 1.0000
10 rbf 0.98
C: 10, Kernel: rbf --> Test Accuracy = 0.9800
20 linear 0.98
C: 20, Kernel: linear --> Test Accuracy = 0.9800
20 rbf 1.0
C: 20, Kernel: rbf --> Test Accuracy = 1.0000

-----
Best Hyperparameters:
Best C : 1
Best Kernel : linear
Best Accuracy : 1.0000
```

3. (Grid Search CV)

Code:

```
[8] # Create SVM model
✓ 0s svm = SVC()

# Parameter Grid
param_grid = {
    'C' : [1, 10, 20],
    'kernel' : ['linear', 'rbf']
}

# Apply GridSearchCV
grid_search = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy'
)

# Train the model
grid_search.fit(X_train, y_train)

# Create DataFrame of results
results = pd.DataFrame(grid_search.cv_results_)
# Display important columns
print("Grid Search Results")
print(results[['param_C', 'param_kernel', 'mean_test_score']])

# Print best parameters
print("\nBest Parameters:")
print(grid_search.best_params_)

# Print best score
print("\nBest Cross Validation Score:")
print(grid_search.best_score_)
```

Output:

```
... Grid Search Results
  param_C param_kernel mean_test_score
0         1         linear           0.95
1         1          rbf           0.93
2        10         linear           0.93
3        10          rbf           0.94
4        20         linear           0.93
5        20          rbf           0.95

Best Parameters:
{'C': 1, 'kernel': 'linear'}

Best Cross Validation Score:
0.95
```

4. (Randomized Search CV)

Code:

```
[10] ✓ 0s # Apply RandomizedSearchCV
random_search = RandomizedSearchCV(
    estimator=svm,
    param_distributions=param_grid,
    n_iter=5,
    cv=5,
    scoring='accuracy',
    random_state=42
)
# Train the model
random_search.fit(X_train, y_train)
# Create Results DataFrame
results = pd.DataFrame(random_search.cv_results_)
# Display important columns
print("Randomized Search Results")
print(results[['param_C', 'param_kernel', 'mean_test_score']])
# Best Parameters
print("\nBest Parameters:")
print(random_search.best_params_)
# Best Score
print("\nBest Cross Validation Score:")
print(random_search.best_score_)
```

Output:

```
... Randomized Search Results
  param_C param_kernel mean_test_score
0         1         linear           0.95
1         1          rbf           0.93
2        20          rbf           0.95
3        10         linear           0.93
4        20         linear           0.93

Best Parameters:
{'kernel': 'linear', 'C': 1}

Best Cross Validation Score:
0.95
```

5. (Bagging – Random Forest)

Code:

```
[12] ✓ 0s # Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X,y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

# Create Random Forest Model
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42)

# Train the Model
rf_model.fit(X_train, y_train)

# Predict on Test Data
y_pred = rf_model.predict(X_test)

# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)

# Print Accuracy
print("Random Forest Accuracy:", accuracy)
```

Output:

```
... Random Forest Accuracy: 0.9
```

6. (Boosting – AdaBoost & Gradient Boosting)

Code & Output:

```
[18] ✓ 0s # AdaBoost Classifier
ada_model = AdaBoostClassifier(
    n_estimators=100,
    random_state=42
)
ada_model.fit(X_train, y_train)
y_pred_ada = ada_model.predict(X_test)
ada_accuracy = accuracy_score(y_test, y_pred_ada)
print("AdaBoost Accuracy:", ada_accuracy)

# Gradient Boosting Classifier
gb_model = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)
gb_model.fit(X_train, y_train)
y_pred_gb = gb_model.predict(X_test)
gb_accuracy = accuracy_score(y_test, y_pred_gb)
print("Gradient Boosting Accuracy:", gb_accuracy)

... AdaBoost Accuracy: 0.9333333333333333
Gradient Boosting Accuracy: 0.9666666666666667
```

7. (Boosting - XGBoost)

Code & Output:

```
[21]
✓ 0s # Create XGBoost Model
xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
    eval_metric='mlogloss'
)
# Train the model
xgb_model.fit(X_train, y_train)
# Predict on Test Data
y_pred_xgb = xgb_model.predict(X_test)
# Calculate Accuracy
accuracy = accuracy_score(y_test, y_pred)
# Print Accuracy
print("XGBoost Accuracy:", accuracy)

... XGBoost Accuracy: 0.9
```

8. (Hyperparameter Tuning on Random Forest)

Code:

```
[22]
✓ 7s # Create Random Forest model
rf_model = RandomForestClassifier(random_state=42)

# Parameter Grid
param_grid = {
    'n_estimators': [50, 100, 150],
    'max_depth': [3, 5, 7]
}
# Apply GridSearchCV
grid_search = GridSearchCV(
    estimator=rf_model,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy'
)
# Train the model
grid_search.fit(X_train, y_train)
# Display Results DataFrame
results = pd.DataFrame(grid_search.cv_results_)
print("Grid Search Results")
print(results[['param_n_estimators',
               'param_max_depth',
               'mean_test_score']])

# Best Parameters
print("\nBest Parameters:")
print(grid_search.best_params_)
# Best Score
print("\nBest Cross Validation Score:")
print(grid_search.best_score_)
```

Output:

```
... Grid Search Results
      param_n_estimators  param_max_depth  mean_test_score
0                      50                3         0.958333
1                      100               3         0.958333
2                      150               3         0.958333
3                      50                5         0.950000
4                      100               5         0.950000
5                      150               5         0.950000
6                      50                7         0.950000
7                      100               7         0.950000
8                      150               7         0.950000

Best Parameters:
{'max_depth': 3, 'n_estimators': 50}

Best Cross Validation Score:
0.9583333333333334
```

9. (Complete Model Comparison)

Code:

```
results = {}

svm = SVC(
    C=1,
    kernel='linear'
)
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)
results['SVM'] = accuracy_score(y_test, svm_pred)

rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
results['Random Forest'] = accuracy_score(y_test, rf_pred)

ada = AdaBoostClassifier(
    n_estimators=100,
    random_state=42
)
ada.fit(X_train, y_train)
ada_pred = ada.predict(X_test)
results['AdaBoost'] = accuracy_score(y_test, ada_pred)

gb = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)
```

```

)
ada.fit(X_train, y_train)
ada_pred = ada.predict(X_test)
results['AdaBoost'] = accuracy_score(y_test, ada_pred)

gb = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    random_state=42
)
gb.fit(X_train, y_train)
gb_pred = gb.predict(X_test)
results['Gradient Boosting'] = accuracy_score(y_test, gb_pred)

xgb = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
    eval_metric='mlogloss'
)
xgb.fit(X_train, y_train)
xgb_pred = xgb.predict(X_test)
results['XGBoost'] = accuracy_score(y_test, xgb_pred)

comparison = pd.DataFrame(
    results.items(),
    columns=['Model', 'Accuracy']
)
print("\nModel Comparison:")
print(comparison)

```

Output:

```

Model Comparison:
   Model  Accuracy
0      SVM  1.000000
1  Random Forest  0.900000
2    AdaBoost  0.933333
3 Gradient Boosting  0.966667
4     XGBoost  0.933333

Best Model:
Model : SVM
Accuracy : 1.0

```

10. (Mini Project – Full Pipeline)

Code:

Part 1:

Step 1: Import Libraries


```
[1]
✓ 1s
import pandas as pd
import numpy as np
import joblib
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
```

Step 2 : Load Dataset:

```
[2]
✓ 0s
df = pd.read_csv("heart - heart.csv")
print(df.head())
```

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	\
0	40	M	ATA	140	289	0	Normal	172	
1	49	F	NAP	160	180	0	Normal	156	
2	37	M	ATA	130	283	0	ST	98	
3	48	F	ASY	138	214	0	Normal	108	
4	54	M	NAP	150	195	0	Normal	122	

	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	N	0.0	Up	0
1	N	1.0	Flat	1
2	N	0.0	Up	0
3	Y	1.5	Flat	1
4	N	0.0	Up	0

Step 3: Check Dataset:

```
[3]
✓ 0s
print(df.shape)
print(df.info())
print(df.isnull().sum())
print(df.describe())
```

```
... (918, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
#   Column              Non-Null Count  Dtype
---  ---
0   Age                 918 non-null   int64
1   Sex                 918 non-null   object
2   ChestPainType       918 non-null   object
3   RestingBP           918 non-null   int64
4   Cholesterol          918 non-null   int64
5   FastingBS           918 non-null   int64
6   RestingECG          918 non-null   object
7   MaxHR               918 non-null   int64
8   ExerciseAngina      918 non-null   object
9   Oldpeak             918 non-null   float64
10  ST_Slope            918 non-null   object
11  HeartDisease        918 non-null   int64
dtypes: float64(1), int64(6), object(5)
```

```

dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
None
Age          0
Sex          0
ChestPainType 0
RestingBP    0
Cholesterol  0
FastingBS    0
RestingECG   0
MaxHR        0
ExerciseAngina 0
Oldpeak      0
ST_Slope     0
HeartDisease 0
dtype: int64

      Age  RestingBP  Cholesterol  FastingBS  MaxHR  \
count  918.000000  918.000000   918.000000  918.000000  918.000000
mean    53.510893  132.396514   198.799564    0.233115  136.809368
std      9.432617   18.514154   109.384145    0.423046   25.460334
min     28.000000    0.000000    0.000000    0.000000   60.000000
25%     47.000000  120.000000   173.250000    0.000000  120.000000
50%     54.000000  130.000000   223.000000    0.000000  138.000000
75%     60.000000  140.000000   267.000000    0.000000  156.000000
max     77.000000  200.000000   603.000000    1.000000  202.000000

      Oldpeak  HeartDisease
count  918.000000  918.000000
mean     0.887364    0.553377
std     1.066570    0.497414
min    -2.600000    0.000000
25%     0.000000    0.000000
50%     0.600000    1.000000

```

Step 4: Features & Target

```

[6] ✓ Os
print(df.columns.tolist())

['Age', 'Sex', 'ChestPainType', 'RestingBP', 'Cholesterol', 'FastingBS', 'RestingECG', 'MaxHR', 'ExerciseAngina', 'Oldpeak', 'ST_Slope', 'HeartDisease']

```

Step 5: Train-Test Split

```

[26] ✓ Os
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y)

```

Step 6: Feature Scaling

```

[32] ✓ Os
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

```

Step 7: Manual Hyperparameter Search (SVM)

```
[34] ✓ 0s C_values = [1, 10, 20]
        kernels = ["linear", "rbf"]

        best_accuracy = 0
        best_C = None
        best_kernel = None

        print("Manual Hyperparameter Search\n")

        for c in C_values:
            for kernel in kernels:
                model = SVC(C=c, kernel=kernel)
                model.fit(X_train, y_train)
                prediction = model.predict(X_test)
                accuracy = accuracy_score(y_test, prediction)
                print(f"C: {c}, Kernel: {kernel}, Accuracy: {accuracy:.4f}")

                if accuracy > best_accuracy:
                    best_accuracy = accuracy
                    best_C = c
                    best_kernel = kernel

        print("\nBest Parameters")
        print("C =", best_C)
        print("Kernel =", best_kernel)
        print("Best Accuracy =", best_accuracy)

        print("Best Accuracy =", best_accuracy)

        Manual Hyperparameter Search

        C: 1, Kernel: linear, Accuracy: 0.8750
        C: 1, Kernel: rbf, Accuracy: 0.9022
        C: 10, Kernel: linear, Accuracy: 0.8750
        C: 10, Kernel: rbf, Accuracy: 0.8967
        C: 20, Kernel: linear, Accuracy: 0.8750
        C: 20, Kernel: rbf, Accuracy: 0.8804

        Best Parameters
        C = 1
        Kernel = rbf
        Best Accuracy = 0.9021739130434783
```

Part 2:

Step 1. Import Libraries

```
[35] ✓ 0s from sklearn.model_selection import GridSearchCV
        from sklearn.model_selection import RandomizedSearchCV
        from sklearn.svm import SVC
        import pandas as pd
```

Step 2. Create SVM Model

```
[36] ✓ 0s svm = SVC()
```

Step 3. Parameter Grid

```
[37] ✓ 0s param_grid = {
        'C': [1, 10, 20],
        'kernel': ['linear', 'rbf']
    }
```

Step 4. GridSearchCV

```
[38] 3s ▶ grid_search = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    cv=5,
    scoring='accuracy',
)
grid_search.fit(X_train, y_train)
```

GridSearchCV ⓘ ?

best_estimator_: SVC

SVC ?

Step 5. Display Results

```
[39] 0s ▶ grid_results = pd.DataFrame(grid_search.cv_results_)
print(grid_results[[
    'param_C',
    'param_kernel',
    'mean_test_score'
]])
```

	param_C	param_kernel	mean_test_score
0	1	linear	0.854152
1	1	rbf	0.862333
2	10	linear	0.852791
3	10	rbf	0.843286
4	20	linear	0.851430
5	20	rbf	0.828283

Step 6. Best Parameters

```
[40] 0s ▶ print("\nBest Parameters:")
print(grid_search.best_params_)

print("\nBest Accuracy:")
print(grid_search.best_score_)
```

Best Parameters:
{'C': 1, 'kernel': 'rbf'}

Best Accuracy:
0.8623334265212934

Step 7. RandomizedSearchCV

```
[41] ✓ 4s random_search = RandomizedSearchCV(
    estimator=svm,
    param_distributions=param_grid,
    n_iter=10,
    cv=5,
    scoring='accuracy',
    random_state=42
)
random_search.fit(X_train, y_train)
```

... /usr/local/lib/python3.12/dist-packages/sklearn/model_selection/_search.py:317: UserWarning: ...

RandomizedSearchCV ⓘ ?

best_estimator_: SVC

SVC ⓘ ?

Step 8. Display Results

```
[42] 0s random_results = pd.DataFrame(random_search.cv_results_)
print(random_results[['param_C',
    'param_kernel',
    'mean_test_score']])
```

	param_C	param_kernel	mean_test_score
0	1	linear	0.854152
1	1	rbf	0.862333
2	10	linear	0.852791
3	10	rbf	0.843286
4	20	linear	0.851430
5	20	rbf	0.828283

Step 9. Best Parameters

```
[43] ✓ 0s print("\nBest Parameters:")
print(random_search.best_params_)

print("\nBest Accuracy:")
print(random_search.best_score_)
```

Best Parameters:
{'kernel': 'rbf', 'C': 1}

Best Accuracy:
0.8623334265212934

Step 10. Compare Grid Search & Random Search

```
[44]
✓ 0s
print("\nComparison")
print("-----")

print("Grid Search")
print("Best Parameters :", grid_search.best_params_)
print("Best Score      :", grid_search.best_score_)
print()
print("\nRandom Search")
print("Best Parameters :", random_search.best_params_)
print("Best Score      :", random_search.best_score_)

...

Comparison
-----

Grid Search
Best Parameters : {'C': 1, 'kernel': 'rbf'}
Best Score      : 0.8623334265212934

Random Search
Best Parameters : {'kernel': 'rbf', 'C': 1}
Best Score      : 0.8623334265212934
```

Part 3:

Step 1. Best SVM (Grid Search)

```
[48]
✓ 0s
best_svm = grid_search.best_estimator_
best_svm.fit(X_train, y_train)
svm_pred = best_svm.predict(X_test)
svm_accuracy = accuracy_score(y_test, svm_pred)
results["SVM"] = svm_accuracy
models["SVM"] = best_svm
```

Step 2. Random Forest

```
[51]
✓ 0s
rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)
results["Random Forest"] = rf_acc
models["Random Forest"] = rf
```

Step 3. AdaBoost

```
[52]
✓ 0s
ada = AdaBoostClassifier(
    n_estimators=100,
    random_state=42
)
ada.fit(X_train, y_train)
ada_pred = ada.predict(X_test)
ada_acc = accuracy_score(y_test, ada_pred)
results["AdaBoost"] = ada_acc
models["AdaBoost"] = ada
```

Step 4. Gradient Boosting

```
[53]
✓ 0s      gb = GradientBoostingClassifier(
            n_estimators=100,
            learning_rate=0.1,
            random_state=42
        )
        gb.fit(X_train, y_train)
        gb_pred = gb.predict(X_test)
        gb_acc = accuracy_score(y_test, gb_pred)
        results["Gradient Boosting"] = gb_acc
        models["Gradient Boosting"] = gb
```

Step 5. XGBoost

```
[54]
✓ 0s      xgb = XGBClassifier(
            n_estimators=100,
            learning_rate=0.1,
            max_depth=3,
            random_state=42,
            eval_metric="logloss"
        )
        xgb.fit(X_train, y_train)
        xgb_pred = xgb.predict(X_test)
        xgb_acc = accuracy_score(y_test, xgb_pred)
        results["XGBoost"] = xgb_acc
        models["XGBoost"] = xgb
```

Comparison Table

```
[55]
✓ 0s      comparison = pd.DataFrame(
            results.items(),
            columns=["Model", "Accuracy"]
        )
        comparison = comparison.sort_values(by="Accuracy", ascending=False)
        print(comparison)
```

	Model	Accuracy
0	SVM	0.902174
4	XGBoost	0.896739
2	AdaBoost	0.891304
3	Gradient Boosting	0.875000
1	Random Forest	0.869565

Best Model

```
▶ best_model_name = comparison.iloc[0]["Model"]
  best_accuracy = comparison.iloc[0]["Accuracy"]
  print("Best Model:", best_model_name)
  print("Best Accuracy:", best_accuracy)

... Best Model: SVM
  Best Accuracy: 0.9021739130434783
```

Save Best Model

```
[57] ✓ Os ▶ best_model = models[best_model_name]
      joblib.dump(best_model, "best_model.pkl")
      joblib.dump(scaler, "scaler.pkl")
      print("Best Model Saved Successfully.")
      ▼ ... Best Model Saved Successfully.
```

Part 4

```
import streamlit as st
import pandas as pd
import joblib

# Load Model and Scaler
model = joblib.load("models/best_model.pkl")
scaler = joblib.load("models/scaler.pkl")

st.set_page_config(page_title="Heart Disease Prediction", layout="centered")

st.title("Heart Disease Prediction")
st.write("Enter patient details below.")

# User Inputs
age = st.number_input("Age", 18, 100, 40)

sex = st.selectbox("Sex", ["M", "F"])

chest_pain = st.selectbox(
    "Chest Pain Type",
    ["ATA", "NAP", "ASY", "TA"]
)

resting_bp = st.number_input("Resting Blood Pressure", 80, 250, 120)

cholesterol = st.number_input("Cholesterol", 0, 700, 200)

fasting_bs = st.selectbox("Fasting Blood Sugar", [0, 1])

resting_ecg = st.selectbox(
    "Resting ECG",
    ["Normal", "ST", "LVH"]
)

max_hr = st.number_input("Maximum Heart Rate", 60, 220, 150)

exercise_angina = st.selectbox(
    "Exercise Angina",
    ["N", "Y"]
)
```



```

)

oldpeak = st.number_input(
    "Oldpeak",
    value=1.0,
    step=0.1
)

st_slope = st.selectbox(
    "ST Slope",
    ["Up", "Flat", "Down"]
)

# Predict Button
if st.button("Predict"):

    # Create Input DataFrame
    input_df = pd.DataFrame({
        "Age": [age],
        "RestingBP": [resting_bp],
        "Cholesterol": [cholesterol],
        "FastingBS": [fasting_bs],
        "MaxHR": [max_hr],
        "Oldpeak": [oldpeak],
        "Sex_M": [1 if sex=="M" else 0],
        "ChestPainType_ATA": [1 if chest_pain=="ATA" else 0],
        "ChestPainType_NAP": [1 if chest_pain=="NAP" else 0],
        "ChestPainType_TA": [1 if chest_pain=="TA" else 0],
        "RestingECG_Normal": [1 if resting_ecg=="Normal" else 0],
        "RestingECG_ST": [1 if resting_ecg=="ST" else 0],
        "ExerciseAngina_Y": [1 if exercise_angina=="Y" else 0],
        "ST_Slope_Flat": [1 if st_slope=="Flat" else 0],
        "ST_Slope_Up": [1 if st_slope=="Up" else 0]
    })

    # Scale Input
    input_scaled = scaler.transform(input_df)

    # Prediction
    prediction = model.predict(input_scaled)

    if prediction[0] == 1:
        st.error(" Heart Disease Detected")
    else:
        st.success(" No Heart Disease Detected")

```

Output:

Heart Disease Prediction

Enter patient details below.

Age

40

- +

Sex

M

▼

M

F

Chest Pain Type

ATA

▼

ATA

NAP

ASY

TA

Resting Blood Pressure

140

- +

Cholesterol

289

- +

Fasting Blood Sugar

0

▼

0

1

Resting ECG

Normal

▼

Normal

ST

LVH

Maximum Heart Rate

172

- +

Exercise Angina

N

▼

N

Y

Exercise Angina

N

Oldpeak

0.00

ST Slope

Up

Up

Flat

Down

ST Slope

Up

Predict

No Heart Disease Detected